

📁 工作经历

杭州磐太科技有限公司

2026.3.9-至今
java后端实习生

项目：古树名木普查（GSDC）

- 解决多巡检员并发提交时编号冲突问题，评估 Redis INCR、数据库序列、Advisory Lock 三种方案后选择 Advisory Lock 实现行级互斥生成，在事务内完成编号分配，避免引入额外中间件依赖；
- 针对外业弱网场景下数据提交丢失的投诉问题，设计草稿/提交分层校验策略：草稿仅校验必填项允许中途保存，提交时执行完整业务规则校验，消除弱网导致的数据丢失风险；
- 主导省级编号字段的端到端改造，沿DDD四层架构（DTO → Command → 领域上下文 → MyBatis SQL）补齐写入链路，前端解除只读锁定并新增校验，改造涉及 17 个文件，48 项单元测试全部通过，零回归缺陷上线；
- 定位古树空间查询的全表扫描瓶颈，通过 EXPLAIN 分析执行计划，为 PostGIS 地理字段创建 GIST 空间索引，查询计划从 Sequential Scan 优化为 Index Scan，地图范围查询响应时间从 ~2s 降至 ~200ms；
- 推动DDD分层架构在项目中的落地，将巡检业务规则从 Service 层下沉至领域实体，隔离业务逻辑与基础设施代码，新需求变更时只需修改领域层，减少跨层影响范围；

技术栈：Java ， Spring Boot ， DDD ， PostgreSQL+PostGIS ， MyBatis-Plus ， RabbitMQ ， OAuth2+JWT ， OSS·MinIO·S3

📁 项目经历

DiffGuard

项目背景：基于 Vibe Coding 范式主导开发的 AI 代码审查系统，通过 ReAct Agent + Function Calling 实现多维度自主代码审查，集成 GitHub PR 工作流，覆盖安全漏洞、性能瓶颈、架构缺陷三个审查维度。

项目亮点：

- 设计多 Agent 协同审查架构，构建 Security / Performance / Architecture 三个 ReAct Agent，各 Agent 可在 8 轮推理循环中自主调用代码分析工具获取上下文，审查范围从单文件 Diff 扩展至项目级依赖链分析，单次 PR 平均发现 3-5 个有效问题；
- 设计双编排引擎实现审查策略的灵活调度，支持 Pipeline 串行与 Multi-Agent 并行两种模式，通过 YAML DSL 外部化策略配置 + StrategyPlanner 动态画像分配权重，新增审查维度仅需 ~80 行代码 + 1 个 Prompt 文件，无需修改核心编排逻辑；
- 设计 Java-to-Python 跨语言 Tool Server，Java 端提供 AST 解析 + 代码知识图谱（BFS 影响集分析）+ RAG 语义检索三层代码理解能力，通过 Session 安全沙箱暴露给 Python Agent 调用，使 LLM 审查基于完整代码上下文而非单文件 Diff，误报率降低约 30%；
- 构建 LLM 调用网关保障服务稳定性，实现限流/熔断/重试三层防护 + 多 Provider 自动降级 + Token 预算控制，JSON 解析成功率从 ~70% 提升至 ~95%，单次审查 Token 消耗降低约 40%；
- 落地端到端工程化链路，GitHub Webhook → RabbitMQ 异步解耦 → 两级缓存（Caffeine + GZIP 文件缓存）跳过重复 Diff 的 LLM 调用，LLM 调用量降低 ~40%，Docker Compose 五服务一键部署，从 PR 提交到审查报告全链路自动化；

技术栈：Java 21 ， LangChain ， FastAPI ， RabbitMQ ， MySQL ， Redis ， Resilience4j ， Docker

Java MCP Server

项目背景：从零实现一个轻量 MCP 协议服务器，不依赖任何 MCP SDK，为 Claude Desktop 等客户端提供代码分析工具。

项目亮点：

- 从零实现 MCP 协议通信层，手写 JSON-RPC 2.0 编解码器处理三类核心方法，基于 AtomicReference 无锁状态机管理连接生命周期，不依赖任何 MCP SDK 即可对接 Claude Desktop 等客户端；
- 设计注解驱动的工具注册框架，通过反射自动完成工具发现、JSON Schema 生成与参数解析，新增工具只需编写一个 @Tool 注解方法，无需修改注册逻辑，开发一个新工具的平均耗时从 ~2h 降至 ~20min；
- 实现 7 个代码分析工具，覆盖代码统计（22 种语言）、依赖分析、复杂度报告、工程化检查（9 项指标）四大类，单次全项目分析耗时 < 5s；
- 设计三层异常分类体系（协议层 / 工具层 / 传输层），统一转换为 JSON-RPC 2.0 标准错误码，客户端可根据错误码精准定位故障层级；

技术栈：Java 17 ， Jackson ， Java Reflection API ， PDFBox ， SLF4J ， JUnit 5 ， Maven

🔧 专业技能

技术栈

熟悉 springboot，MyBatis 等主流框架技术，熟悉 SpringIOC、AOP 和 SpringMVC 等；
在实习项目中接触了DDD分层架构，理解Domain层与应用层的职责划分；
掌握 MySQL 数据库，了解事务管理，SQL 优化，索引设计；熟悉常用数据结构与算法；
熟悉 Java 集合框架，多线程等相关内容，了解锁机制，Threadlocal，IO 模式以及常见设计模式；
了解Redis基础数据结构的使用场景；在项目中使用Redis做接口鉴权Token缓存，了解缓存过期策略和淘汰机制；
了解 JVM 内存结构、类加载机制、垃圾回收机制；熟悉常用的数据结构，了解算法设计基础；
在实习项目中使用RabbitMQ处理异步任务分发，了解消息确认机制（ACK）和基本的消息可靠性保障方式；
具有 LangChain ReAct Agent 开发经验；理解 MCP 协议规范，从零实现过 MCP Server；